MAGIK v1.0.4 Release Runbook

How to tag, deploy and verify a release, written from the v1.0.4 release in AWS account 266901698137

> **Prepared:** 28 October 2026 · **Repo:** vjkarthik98/MULTIMODAL-AGENTIC-RAG-INTEGRATED-KNOWLEDGE-AI-ASSISTANT
> **Current release:** v1.0.4 (tag on `58bd758`, `main` at `b876071`) · **Next version:** written as `X.Y.Z` below
> **Staging:** i-067fa0abb13ed6481 · **Production:** i-022cdb0f18161a280 · **Region:** us-east-1
> **Replaces:** MAGIK_v1.0.2_Release_Runbook.pdf (12 September 2026)

What changed since the v1.0.2 runbook

The v1.0.2 runbook was written around one release and one broken staging box. These are the lessons from v1.0.3 and v1.0.4 that change how a release is run now.

1. **The tag must sit on a commit whose CI and Security runs have finished.** The first `v1.0.4` tag sat on a commit that had no CI run. `wait-for-ci-green` polled that exact commit for 20 minutes and failed. The tag was moved to `58bd758`. Step 2 checks this before you tag.
2. **GitHub deletes the staging runner after 14 days offline.** Staging only starts on a tagged deploy, so any gap of more than 14 days between releases leaves no runner, and `tier2-staging-gate` queues for up to 24 hours while the GPU box keeps billing. The v1.0.4 gate hit this on 27 October. Step 3 checks it.
3. **The staging BM25 problem from v1.0.2 is gone.** The v1.0.4 gate passed its BM25 preflight, so the index exists on staging. It lives on the root volume, so it is blank again after any rebuild (recovery is under "If something goes wrong").
4. **Cold start is about seven minutes of model loading, and the LLM now starts on the GPU or not at all.** If the GPU is missing, `start_server.py` retries and then exits non-zero so the container restarts and the failure is visible, instead of serving silently on CPU. While models load, Caddy shows a "MAGIK AI is starting up" page.
5. **A suspended Qdrant Cloud cluster blocks a release.** It was found suspended on 27 October; resuming it fixed retrieval with the data intact. Check it before tagging.
6. `release.yml` **does not load on GitHub right now**, so the tag is pushed by hand (Step 4), as it was for v1.0.2 and v1.0.4. Details are in the open items.

Before you start

These were read from live GitHub and AWS on 28 October 2026. They are the assumptions the rest of this document rests on.

Check	Result
`origin/main` and VERSION	b876071, VERSION and pyproject.toml both 1.0.4
Tag `v1.0.4`	exists, on 58bd758
GitHub Release for v1.0.4	missing (latest Release on GitHub is still v1.0.1)
Staging box	stopped, g6e.xlarge, us-east-1c
Production box	stopped, g6e.xlarge, us-east-1c, Elastic IP 34.199.163.242
`magik-staging-runner` (label `staging-gpu`)	registered, offline (normal while the box is stopped)
`magik-prod-runner` (label `gpu`)	NOT registered
`SELF_HOSTED_STAGING_GPU_RUNNER`	true (correct, the runner exists)
`SELF_HOSTED_GPU_RUNNER`	true (wrong while the production runner is missing)
Deploy roles (both variables)	arn:aws:iam::266901698137:role/magik-deploy-role
Lambdas	`magik-idle-stop` and `magik-wake-gateway` present
Staging BM25 index	present as of the v1.0.4 gate pass; not re-read, the box is stopped
Qdrant Cloud cluster	not checked from here; check in the Qdrant Cloud console

> **Two checks before every tag.** Both failed during v1.0.4 and both cost a retry.
>
> - A staging runner is registered (Step 3).
> - The Qdrant Cloud cluster is running, not suspended.

Sequence at a glance

Step	Action	Expected outcome
1	Land the release on main	VERSION, pyproject.toml and CHANGELOG at X.Y.Z on main
2	Confirm CI and Security are green on that commit	two completed, successful runs
3	Check the staging runner	magik-staging-runner registered
4	Create and push the tag	CD pipeline starts
5	Publish the GitHub Release	vX.Y.Z listed with notes
6	Watch the CD pipeline	gate passes, production deploys
7	Verify, and confirm staging stopped	release complete, no billing left on

Steps

Step 1: Land the release on main

Only `VERSION` and `pyproject.toml` carry the version (the app reads it from pyproject.toml at runtime). Add a `## [X.Y.Z] - YYYY-MM-DD` section to `CHANGELOG.md` in the same change. Work goes to `development` first and reaches `main` only through a pull request.

```
git checkout development
git pull origin development
# edit VERSION, pyproject.toml (version = "X.Y.Z") and CHANGELOG.md
git add VERSION pyproject.toml CHANGELOG.md
git commit -m "chore(release): vX.Y.Z"
git push origin development
gh pr create --base main --head development --title "Release vX.Y.Z"
```

Merge when the required checks pass: lint + typecheck + unit (py3.10 and py3.11), tier1-retrieval, detect-secrets, Bandit SAST, pip-audit and the dependency license scan.

Step 2: Confirm CI and Security are green on that exact commit

`cd.yml` will not build an unverified commit. Check the same thing it checks.

```
git checkout main
git pull origin main
SHA=$(git rev-parse HEAD)
gh run list --commit "$SHA" --json name,status,conclusion \
  --jq '.[] | select(.name=="CI" or .name=="Security") | [.name,.status,.conclusion] | @tsv'
```

Expect two rows, both **completed success**. If a row is missing, do not tag yet. Some commits have no CI run at all, for example a docs-only push to main, which `ci.yml` ignores. The first v1.0.4 tag sat on a commit like that.

Step 3: Check the staging runner

```
gh api repos/:owner/:repo/actions/runners --jq '.runners[] | [.name,.status] | @tsv'
```

You want `magik-staging-runner`. **Offline is fine** while the box is stopped. An empty list means GitHub deleted it, and the gate would queue forever. Re-register it:

```
# 1. start staging and wait for SSM to report Online
aws ec2 start-instances --profile magik-admin-new --region us-east-1 --instance-ids
i-067fa0abb13ed6481

# 2. mint a one-hour registration token on your machine
gh api -X POST repos/:owner/:repo/actions/runners/registration-token --jq .token

# 3. on the box, through SSM (AWS-RunShellScript), run as root:
systemctl stop "actions.runner.*"
cd /opt/actions-runner
rm -f .runner .credentials .credentials_rsaparams
RUNNER_ALLOW_RUNASROOT=1 ./config.sh --unattended --replace \
  --url https://github.com/{OWNER_REPO} \
  --token <TOKEN> --name magik-staging-runner --labels staging-gpu
systemctl start "actions.runner.*"
```

The service runs as root and the runner folder is owned by numeric uid 1001 with no passwd entry, so `sudo -u` the folder owner fails with "unknown user". Stop staging again when the runner shows online.

Step 4: Create and push the tag

```
git checkout main
git pull origin main
git tag -a vX.Y.Z -m "<Release title>"
git push origin vX.Y.Z
```

Use `-a` (annotated), like the existing tags. The push triggers `cd.yml`, which fires on `push: tags: ["v*"]`. The tag must be on the commit you checked in Step 2.

Step 5: Publish the GitHub Release

Without this the version is a bare tag. v1.0.2 and v1.0.4 are both bare tags today.

```
gh release create vX.Y.Z \
  --title "vX.Y.Z - <Release title>" \
  --notes "$(awk '/^## \[X.Y.Z\]/{f=1;next} /^## \[/{f=0} f' CHANGELOG.md)"
```

The `awk` pulls the `[X.Y.Z]` section out of `CHANGELOG.md`.

Step 6: Watch the CD pipeline

```
gh run watch $(gh run list --workflow=cd.yml --limit 1 \
  --json databaseId --jq '.[0].databaseId')
```

Job order and what to expect:

Job	What it does	Limits
`wait-for-ci-green`	polls the tagged commit for CI and Security to finish green	20 min per workflow, 25 min job
`build-push`	builds the image once and pushes it to GHCR	
`deploy-staging`	wakes staging, checks disk (30 GB free), pulls secrets from SSM, swaps the container, waits for /health	health check 20 min, SSM 40 min
`tier2-staging-gate`	runs on the staging-gpu runner: container healthy, BM25 file present, full suite, auto-rollback on a retrieval breach	180 min
`stop-staging`	stops the staging box (runs even when earlier jobs fail)	if: always()
`promote-production`	only if the gate passed: wakes production, waits for SSM, deploys the same image, syncs monitoring	health check 20 min

If `deploy-staging` fails its health check it removes the new container and restores `magik-staging-previous`, so staging stays on the last good image. Read the last 80 log lines in the job output. A line saying the GPU was not used means the box lost its GPU, not that the model is slow. Re-run just the failed jobs once the cause is fixed:

```
gh run rerun <run-id> --failed
```

> **If tier2-staging-gate sits queued,** no runner is registered. Cancel the run, stop staging by hand (it bills about $1.86 per hour), do Step 3, then re-run.

Step 7: Verify and confirm staging is stopped

```
curl -sS https://magik.vk-ai.online/health
curl -sS https://magik.vk-ai.online/version
gh release view vX.Y.Z
aws ec2 describe-instances --profile magik-admin-new --region us-east-1 \
  --instance-ids i-067fa0abb13ed6481 i-022cdb0f18161a280 \
  --query 'Reservations[].Instances[].{Name:Tags[?Key==`Name`]|[0].Value,State:State.Name}' \
  --output table
```

`/version` must report X.Y.Z. Production stays running after promotion and the `magik-idle-stop` Lambda stops it when idle. **Staging must end up stopped.** `stop-staging` does it, but check after re-runs or manual starts: the idle-stop Lambda only watches `magik-prod`, so nothing else ever stops staging. A forgotten box is about $45 a day.

```
aws ec2 stop-instances --profile magik-admin-new --region us-east-1 --instance-ids
i-067fa0abb13ed6481
```

After promotion the pipeline also syncs the monitoring stack. That step is non-blocking: a failure there leaves the app deployed and sends a Slack message.

> **Not part of this pipeline.** `cd.yml` ships the app image and the monitoring stack only. The Lambdas (`magik-idle-stop` , `magik-wake-gateway`) and Terraform or IAM changes are deployed separately. When running `deploy_lambdas.sh` , pass `KUMA_PUSH_URL` explicitly, because the script defaults it to empty and silently disables Uptime Kuma on both Lambdas. Run it from AWS CloudShell rather than Git Bash.

If something goes wrong

The tag is on the wrong commit, or wait-for-ci-green failed

```
git push origin :refs/tags/vX.Y.Z      # delete remote tag
git tag -d vX.Y.Z                      # delete local tag
gh release delete vX.Y.Z --yes         # only if the Release was created
```

Fix the cause, redo Step 2, then tag again. Deleting a tag does not undo a deploy that already reached production, so check what `promote-production` did first.

The gate fails on the BM25 preflight

`tier2-staging-gate` needs `data/users/<EVAL_USER_ID>/bm25_index/bm25.pkl` inside the container. The index lives on the box's root volume, so a rebuilt box starts without it. The user id is the repo variable `EVAL_USER_ID` (see `gh variable list`). Start staging if `stop-staging` stopped it, then rebuild through SSM:

```
aws ssm send-command \
  --profile magik-admin-new --region us-east-1 \
  --instance-ids i-067fa0abb13ed6481 \
  --document-name "AWS-RunShellScript" \
  --comment "rebuild staging BM25 index" \
  --timeout-seconds 600 \
  --parameters 'commands=[
    "docker start magik-staging-current || true",
    "docker exec magik-staging-current python3.12 -m app.retrieval.bm25_retriever --user_id
<EVAL_USER_ID>",
    "ls -la /opt/magik/data/users/<EVAL_USER_ID>/bm25_index/"
  ]' \
  --query 'Command.CommandId' --output text
```

Read the result with `aws ssm get-command-invocation` and the returned command id. Success is a listing that shows `bm25.pkl` . It is cheap: it reads payloads back from Qdrant Cloud, with no GPU work and no re-embedding. Then `gh run rerun <run-id> --failed` .

Retrieval returns nothing although the index exists

Check the Qdrant Cloud cluster. On 27 October it was found suspended; resuming it from the console fixed retrieval with the data intact (1236 points for the eval tenant).

Production needs rolling back

Every deploy keeps the previous container, renamed rather than removed. On the production box:

```
docker rm -f magik-current
docker rename magik-previous magik-current
docker start magik-current
```

Staging uses `magik-staging-current` and `magik-staging-previous` the same way. A retrieval-section breach in the gate already rolls staging back by itself, so production never sees that image.

Monitoring sync fails although the app deployed

The step does a `git fetch` and `git reset` in the production checkout. On 27 October it failed silently because git refused the folder as owned by a different user; a system-level `safe.directory` entry on the production box fixed it. Re-check that entry if the box is ever rebuilt.

Open items, not blocking a release

Item	Detail
`release.yml` does not load	Every push creates a failed run for it with "workflow file issue", and none of its 173 runs was a manual dispatch. The likely cause is line 301, a comment inside a `run:` block that contains a literal `${{ }}`; GitHub evaluates that as an empty expression. Once fixed, its prepare and tag modes replace Steps 1, 4 and 5. Until then the manual path above is the one that works.
Missing GitHub Releases	v1.0.2 and v1.0.4 are tags with no Release entry, so GitHub still shows v1.0.1 as Latest. v1.0.3 was never tagged; its changes shipped inside v1.0.4 (CHANGELOG has all three sections).
`magik-prod-runner` missing	Only the staging runner is registered, yet `SELF_HOSTED_GPU_RUNNER` is true. The `[self-hosted, gpu]` job in `eval-gate.yml` queues until timeout instead of skipping. Register the runner on the production box (same recipe as Step 3, label `gpu`) or set the variable to false.
Staging runner expiry	If staging is not started for 14 days, GitHub deletes the runner again. Step 3 is the check; there is no automatic fix.
Quality report scores	The first production run of `run_quality_report.sh` (27 October) completed without an out-of-memory kill, but the scores are not publishable yet: some RAGAS rows are dropped while the reported row count hides it, DeepEval lower-is-better metrics are not inverted, and the judge sometimes returns malformed JSON. These are v1.0.5 candidates.

Reference values

Name	Value
AWS account	266901698137
CLI profile	`magik-admin-new`
Region	us-east-1
Staging instance	i-067fa0abb13ed6481 (g6e.xlarge, us-east-1c, tag `Name=magik-staging`)
Production instance	i-022cdb0f18161a280 (g6e.xlarge, us-east-1c, tag `Name=magik-prod`)
Production Elastic IP	34.199.163.242
Deploy role (both variables)	`arn:aws:iam::266901698137:role/magik-deploy-role`
Repo variables	`AWS_DEPLOY_ROLE_ARN`, `AWS_STAGING_DEPLOY_ROLE_ARN`, `AWS_REGION`, `EC2_INSTANCE_TAG`, `EC2_STAGING_INSTANCE_TAG`, `EVAL_USER_ID`, `SELF_HOSTED_GPU_RUNNER`, `SELF_HOSTED_STAGING_GPU_RUNNER`
Staging runner	`magik-staging-runner`, label `staging-gpu`
Staging container	`magik-staging-current` / `magik-staging-previous`
Production container	`magik-current` / `magik-previous`
Host paths (both boxes)	`/opt/magik/{.env, .hf_cache, data, logs}`
Public endpoint	https://magik.vk-ai.online
Lambdas	`magik-idle-stop`, `magik-wake-gateway`

Generated from live AWS and GitHub state on 28 October 2026. Instance IDs and IPs change on every rebuild: re-verify with `terraform output` rather than trusting these values after any infrastructure change.